MIT 8103 Advanced Database Systems: Portfolio Summary

Ride-Hailing System | Glory Praise Emmanuel | Student ID: 301917908

GitHub: https://github.com/emmaglorypraise/Emmanuel_Glory_Praise_301917908

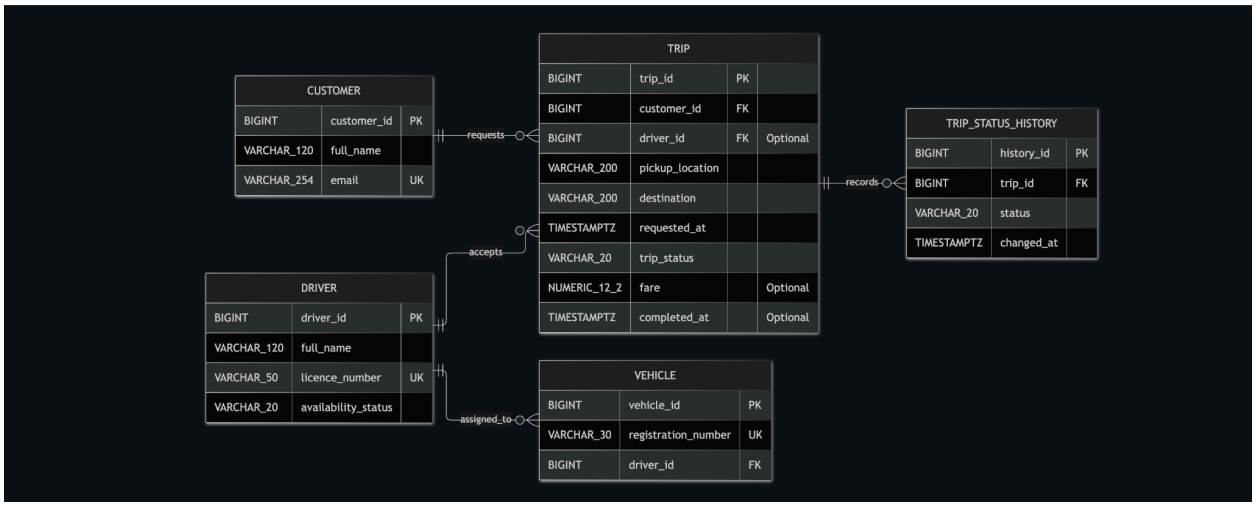
Case study

The selected case study is a fictional, single-city ride-hailing system. It manages customers, drivers, vehicles, trip requests, statuses, and fares. The project uses PostgreSQL for structured operational records and MongoDB for flexible trip activity events, covering design, optimisation, transactions, NoSQL modeling, and a proposed distributed architecture.

Portfolio 1: Database Design and Modelling

I designed a PostgreSQL database with five tables: customer, driver, vehicle, trip, and trip_status_history. I documented the relationships, business rules, and normalisation through 1NF, 2NF, and 3NF. The schema uses primary and foreign keys, NOT NULL, UNIQUE, and CHECK constraints to enforce required values, unique identifiers, valid statuses, non-negative fares, and valid trip-completion details.

The implementation produced 2 customers, 2 drivers, 2 vehicles, 5 trips, and 12 trip-status history records. The invalid insert tests recorded in the repository were rejected as expected, including tests for duplicate emails, invalid foreign keys, missing required values, invalid statuses, negative fares, and invalid trip details. This demonstrates how constraints protect data integrity. Some rules, such as preventing a driver from having multiple active trips, need additional transaction logic.



Portfolio 2: Query Processing and Optimisation

I wrote three queries to retrieve trip details, identify available drivers without a trip in progress, and count completed trips per customer. The queries returned five trips for 28 October 2026,

Daniel Okafor as the only available driver without an ongoing trip, and one completed trip for Ada James compared with zero for Ben Cole.

For optimisation, I created a separate benchmark table with 100,000 synthetic records. I compared a seven-day date-range query before and after adding a B-tree index on requested_at, using EXPLAIN (ANALYZE, BUFFERS) [1]. Both versions returned 1,918 rows.

Measurement	Before index	After index
Scan type	Sequential scan	Index-only scan
Run 1	7.027 ms	0.350 ms
Run 2	5.940 ms	0.260 ms
Run 3	7.339 ms	0.239 ms
Median	7.027 ms	0.260 ms
Buffer hits (run 3)	1,087	5

The index reduced the median execution time in these runs and changed the plan to an index-only scan. The benchmark used synthetic data and three measured runs, so it does not guarantee the same performance in other workloads. Indexes also require storage and maintenance.

Portfolio 3: Transactions and Concurrency

I practised COMMIT and ROLLBACK. Daniel Okafor's availability change from Available to Unavailable was committed; repeating the conditional update returned UPDATE 0. In a separate transaction, I changed Mary's availability and attempted to insert a customer with Ada's existing email. The duplicate-email error caused the transaction to be rolled back, restoring Mary's earlier status and leaving Ada's record unchanged.

For concurrency, two READ COMMITTED sessions tested a row lock on Mary's record. Session B waited while Session A held the lock. However, the saved final output shows Session B returned UPDATE 1, not the expected UPDATE 0. I restarted the test to work through the steps again, but the recorded result does not conclusively demonstrate the intended outcome. This exercise showed the importance of transaction order, row-level locking, and careful concurrent testing [2].

Portfolio 4: NoSQL and Advanced Data Models

I installed MongoDB Community Server 8.0.34 and Mongosh 2.14.1, then created the ride_hailing_activity database and trip_events collection. Four fictional event documents represented TripRequested and TripCompleted activities. The count query returned four documents; Ada's history query returned three events in chronological order; and the nested pickup-location query returned two events for Ikeja. This demonstrated flexible document modeling and nested-field queries [3]. The MongoDB examples were entered separately and were not synchronised automatically with PostgreSQL.

Portfolio 5: Distributed and Cloud Database Exercise

I documented a proposed three-member MongoDB replica set, with members on separate hosts and failure zones. One member would be primary and two secondaries. The discussion covered primary failure, elections, majority write concern, and possible replication lag [4]. I also explained that replication copies data, while sharding distributes records. The four-document collection does not currently require sharding.

I identified authentication, encryption, restricted network access, and tested backups as considerations for a future cloud deployment. This was a proposal only: I did not deploy a replica set, implement sharding, or perform a failover test. MongoDB replication would not synchronise the separate PostgreSQL database.

Evidence

Scripts, explanations and screenshots: P1 design and tests, P2 plans and timings, P3 transactions, P4 events, P5 proposal. Earlier screenshots record earlier database states; Portfolio 3 changes the original trip records.

Individual reflection

I chose the ride-hailing case study because I use these applications and wanted to understand their databases. I used PostgreSQL constraints to improve data accuracy and integrity, and chose a timestamp index because users may remember when a trip happened rather than the driver or vehicle. Managing two concurrent sessions was difficult, and I made several mistakes while testing. I restarted the tests and repeated the steps to obtain fresh measurements, including for query optimisation. The work gave me hands-on understanding of database concepts and a clearer understanding of the differences between SQL and NoSQL.

References

[1] PostgreSQL Global Development Group. PostgreSQL 16: Using EXPLAIN. https://www.postgresql.org/docs/16/using-explain.html

[2] PostgreSQL Global Development Group. PostgreSQL 16: Transaction Isolation. https://www.postgresql.org/docs/16/transaction-iso.html

[3] MongoDB. Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/

[4] MongoDB. Replication. https://www.mongodb.com/docs/manual/replication/

Additional topic-specific references are listed in the relevant portfolio READMEs.

AI-use declaration

I used ChatGPT to explain database concepts and assist with the structure, scripts, analysis, and wording of my portfolio. I did not use other AI tools. I personally reviewed the work, executed and tested every database command, verified the results, and captured screenshots during the exercises. I understand the project and remain responsible for the accuracy of the submitted work.